

Лабораторная работа №3

Реализация серверной логики, разработка API и интеграция с интерфейсом

Цель работы:

Реализовать REST API и алгоритм планирования на стороне сервера, разработать пользовательский интерфейс и выполнить полную интеграцию frontend и backend в работающее веб-приложение.

Формат отчётности:

Документ в формате .docx (Word) + файлы проекта (исходный код).

Отчёт сдаётся в электронном виде. Отчет должен содержать фрагменты исходного кода, связанные с каждым этапом, скриншоты реализованных экранов, примеры запросов к REST API.

Этап 1. Реализация REST API

На данном этапе необходимо **полноценно реализовать программный код** серверной части, обеспечивающий взаимодействие с клиентским приложением через REST-интерфейс.

1.1. Состав реализуемых эндпоинтов

- Реализовать **все эндпоинты**, спроектированные в Лабораторной работе №2.
- При необходимости дополнить перечень методами, требуемыми для поддержки пользовательских сценариев.

1.2. Требования к реализации

- Эндпоинты должны быть **работоспособными** и протестированы (Postman, curl).
- Обрабатывать **корректные запросы** (возвращать статус 200, 201).
- Обрабатывать **ошибочные ситуации**:
 - ресурс не найден — 404;

- неверные входные данные — 400;
 - внутренняя ошибка сервера — 500.
- Реализовать **сохранение и чтение данных из БД** через слой репозитория.
- Применять **сериализацию/десериализацию (DTO)** при обмене данными между клиентом и сервером.

Этап 2. Реализация алгоритма планирования

Необходимо реализовать **алгоритм планирования**, спроектированный в Лабораторной работе №2, в виде **работающего программного кода** на стороне сервера.

2.1. Требования к реализации алгоритма

- Алгоритм оформляется как **отдельный сервисный класс/модуль (Service)**.
- Входные данные алгоритма поступают:
 - из БД (через репозитории);
 - или из параметров запроса к API.
- Результат работы алгоритма сохраняется в БД (например, формируются записи в таблице назначений/расписания).
- Код алгоритма должен быть **читаемым**, содержать комментарии к ключевым блокам логики.

2.2. Связь алгоритма с API

- Создать **отдельный эндпоинт**, запускающий выполнение алгоритма (например, POST /api/planning/run).
- Дополнительно можно интегрировать вызов алгоритма внутрь другого эндпоинта (например, при создании заявки автоматически запускается перепланирование).

2.3. Проверка работоспособности

- Выполнить **ручное тестирование** алгоритма на наборе тестовых данных.
- Убедиться, что результат соответствует ожидаемому (согласно постановке задачи).
- При наличии ошибок — выполнить отладку и исправление.

Этап 3. Разработка пользовательского интерфейса

3.1. Реализация экранов

- Разработать **web-интерфейс** на основе макетов, созданных в ЛР №1 и уточнённых в ЛР №2.
- Минимальный стек: HTML/CSS + JavaScript (чистый JS или с использованием фреймворков, например, React/Vue).
- Интерфейс должен включать **все ключевые экраны**, определённые ранее.

3.2. Требования к интерфейсу

- Интерфейс должен быть **функциональным** (все кнопки, поля, формы работают).
- Навигация между экранами (ссылки, кнопки) должна быть реализована.
- Допускается использование CSS-фреймворков (Bootstrap, Tailwind и др.).

Этап 4. Интеграция frontend и backend

4.1. Подключение API-клиента

- На стороне frontend реализовать **HTTP-запросы** к разработанному API.
- Использовать fetch, axios или аналоги.
- Базовый URL API вынести в конфигурацию (константу).

4.2. Отображение данных

- При загрузке экрана выполнять **GET-запросы** и отображать полученные данные.
- Реализовать **формы** для отправки данных (POST, PUT).
- Обрабатывать **ответы сервера**:
 - при успехе — обновлять интерфейс, показывать уведомление;
 - при ошибке — показывать сообщение об ошибке.

Этап 5. Тестирование интеграции

5.1. Проверка основных эндпоинтов через интерфейс

- Убедиться, что каждый экран корректно получает и отправляет данные.
- Проверить **граничные случаи**:
 - попытка создать объект с пустыми полями;
 - запрос несуществующего объекта;
 - запуск алгоритма при отсутствии данных.

5.2. Исправление дефектов

- Выявленные ошибки должны быть **исправлены**.
- Кодовая база должна находиться в **работоспособном состоянии** к моменту сдачи.